

SnapFeast System Architecture

Brief Introduction to Project

SnapFeast is a mobile application designed to enhance the dining experience through personalized menu recommendations. Utilizing Convolutional Neural Networks (CNN) for facial recognition and collaborative filtering techniques, SnapFeast delivers a highly tailored dining experience by identifying users through facial recognition and recommending dishes based on their previous dining preferences.

Purpose and Scope

The goal of SnapFeast is to provide a seamless, intuitive, and customer-centric dining experience by:

- Identifying customers through facial recognition.
- Analyzing historical dining data to understand preferences and budget.
- Using collaborative filtering to recommend dishes.
- Managing orders and feedback through an interactive user interface.

Definitions and Acronyms

- **CNN (Convolutional Neural Network):** Deep neural networks used for analyzing visual imagery.
- **Facial Recognition:** Technology to identify or verify a person from a digital image or video frame.
- **Collaborative Filtering:** Method to predict user interests by collecting preferences from many users.

- **ORM (Object-Relational Mapping):** Technique for converting data between incompatible type systems using object-oriented programming languages.
- **Flutter:** Open-source UI software development kit created by Google for building natively compiled applications for mobile, web, and desktop from a single codebase.
- **Django:** High-level Python web framework for rapid development and clean design.

System Overview

SnapFeast integrates advanced facial recognition technology and collaborative filtering to provide personalized recommendations. The application includes:

- **User Identification:** Using CNN-based facial recognition.
- **Data Analysis:** Understanding customer preferences and budget constraints.
- **Personalized Recommendations:** Suggesting dishes that align with user preferences.
- **Order Management:** Browsing menus, placing orders, and providing feedback through an intuitive interface.
- **Feedback Integration:** Improving recommendations based on user feedback.

Frontend and UI/UX Information

- **Consistency:** Maintain a consistent design language with cohesive color schemes, typography, and layout.
- **Clarity:** Present information clearly, avoiding clutter and using whitespace effectively.
- **User-Centric:** Focus on an intuitive and seamless user experience.
- **Accessibility:** Ensure accessibility for users with disabilities, using appropriate contrast ratios, alt text, and keyboard navigation.

Pages of the Application

1. Landing Page
2. User Registration and Authentication Page
3. Dashboard (Post-login)
4. Menu and Order Page
5. Recommendations Page
6. Profile Settings Page
7. Order History Page
8. Feedback Page
9. Help and Support Page
10. About Us Page
11. Privacy Policy and Terms of Service Page

Detailed Explanation of Pages

1. **Landing Page:**

- Introduction to SnapFeast.
- Key features and benefits.
- Call-to-action buttons (e.g., "Get Started," "Browse Menu").

2. **User Registration and Authentication Page:**

- User sign-up form.
- User login form.
- Password recovery/reset options.

3. **Dashboard (Post-login):**

- Overview of user activity and preferences.
- Quick access to menu browsing and order placement.

- Links to profile settings, order history, and support.

4. Menu and Order Page:

- Display of the restaurant's menu.
- Options to place orders.
- View of current and past orders.

5. Recommendations Page:

- Personalized dish recommendations.
- Analysis of past preferences and budget.

6. Profile Settings Page:

- User profile information.
- Options to update personal details, change passwords, etc.

7. Order History Page:

- List of past orders.
- Detailed view of each order.

8. Feedback Page:

- Form for submitting feedback about SnapFeast.
- Option to suggest improvements or report issues.

9. Help and Support Page:

- FAQs.

- Contact information for support.
- User guides and tutorials.

10. About Us Page:

- Information about the SnapFeast team.
- Mission and vision statements.

11. Privacy Policy and Terms of Service Page:

- Detailed information on user data privacy and application terms of use.

Backend and Machine Learning Engineering Documentation

API Endpoints

User Registration and Authentication:

- `POST /api/register/`: Register a new user (username, password, email).
- `POST /api/login/`: Authenticate a user and return a JWT token.
- `POST /api/logout/`: Logout user, invalidating the session.
- `POST /api/verify-facial-recognition/`: Verify user via facial recognition.

Menu and Order Management:

- `GET /api/menu/`: Retrieve all menu items.
- `POST /api/order/`: Place a new order.
- `GET /api/order/{order_id}/`: Retrieve specific order details.
- `GET /api/user/orders/`: Retrieve user's order history.

Recommendation System:

- **GET /api/recommendations/**: Get personalized menu recommendations.

User Feedback:

- **POST /api/feedback/**: Submit user feedback.

Authentication for User and Developer Access

User Authentication:

- Uses JWT for secure authentication. Users receive a token upon login, required for accessing protected endpoints.

Developer Access:

- API keys for developers to access specific endpoints, ensuring controlled access and security.

Integration of Machine Learning with Backend

Model Loading:

- Load CNN models for facial recognition and collaborative filtering when the server starts.
- Ensure models are optimized for efficient inference.

Image/Video Processing and Analysis:

- Develop functions for image and video processing.

- Implement endpoints to call ML models for predicting user identification and generating recommendations.

Data Handling:

- Handle input data properly (images/videos).
- Store and manage results in the database for efficient retrieval.

Backend Integration with Frontend

API Documentation:

- Provide detailed API documentation using tools like Swagger or Postman for frontend developers.

Asynchronous Communication:

- Implement asynchronous calls from Flutter to backend endpoints.
- Smooth handling of data loading states and dynamic result display.

Real-time Updates:

- Use technologies like WebSocket for real-time updates, particularly for order status and feedback.

Backend Engineering

Server Setup and Framework:

- Use Django with Django REST Framework for backend development.
- Define API endpoints for various functionalities.

Database Design:

- Use PostgreSQL for database management.
- Design a schema to store user information, menu items, orders, and user preferences.

API Development:

- Develop RESTful API endpoints for:
 - User registration and authentication.
 - Image/video uploads.
 - Analysis requests and responses.
 - Order management.
 - User feedback submissions.

Security:

- Implement authentication and authorization mechanisms (e.g., JWT, OAuth).
- Ensure data encryption and secure communication (HTTPS).

Scalability and Performance Optimization:

- Use asynchronous processing for heavy tasks.
- Implement caching strategies (e.g., Redis) to enhance response times.
- Set up load balancing and auto-scaling to handle varying loads efficiently.

Machine Learning Engineering

Model Training and Optimization:

- Train CNN models on relevant datasets.

- Optimize models using techniques like quantization and knowledge distillation.

Model Serving:

- Use frameworks like TensorFlow Serving, TorchServe, or Flask to serve models.
- Develop endpoints for model inference (e.g., `/predict` for facial recognition and recommendations).

Integration with Backend:

- Develop functions to call ML models from backend endpoints.
- Process input data (images/videos) appropriately for model inference.
- Handle model responses and prepare them for frontend display.

Synchronizing Backend and Frontend

API Documentation:

- Use tools like Swagger or Postman to document API endpoints for frontend developers.

Frontend Integration:

- Ensure the frontend team understands data formats expected by the API.
- Develop asynchronous calls to handle requests and responses smoothly.
- Handle loading states and display results dynamically.

Real-time Communication:

- Implement WebSocket for real-time updates, such as order status and feedback.
- Ensure responsive user experience with real-time interactions.

Example Workflow

1. User Uploads Image/Video:

- Frontend sends the file to the backend endpoint (`/upload`).
- Backend saves the file and initiates analysis using ML models.

2. Backend Analysis:

- Backend calls the ML model for facial recognition and recommendation generation.
- Results are processed, formatted, and stored in the database.

3. Results Display:

- Backend sends analysis results to the frontend.
- Frontend displays user identification and personalized recommendations.

4. Order Placement:

- User places an order through the frontend.
- Backend processes the order and updates the database.

5. Interactive Chat (if applicable):

- User initiates a chat about recommendations or orders.

- Frontend sends chat messages to the backend.
- Backend processes messages and responds with appropriate information.

Ways to Improve Project Idea

1. **User Feedback Integration:** Continuously collect and integrate user feedback to improve the experience and add new features.
2. **Advanced Analytics:** Incorporate more advanced analytics to provide deeper insights into user preferences.
3. **Real-time Analysis:** Enhance the system to provide real-time analysis for live interactions.
4. **Expand Use Cases:** Explore additional use cases for the technology, such as integrating with smart restaurant infrastructure or health monitoring systems.
5. **Mobile Application:** Develop a mobile application to make SnapFeast more accessible and user-friendly on various devices.
6. **API Access:** Provide API access for third-party developers to integrate SnapFeast's capabilities into their applications.
7. **Enhanced Security:** Implement advanced security measures to ensure user data privacy and application security.